

TECHNICAL & PRODUCT DOCUMENTATION

Pozzy

Personal Operating System — a private web app for running one week: calendar, mail triage, tasks, goals, habits, hours, study and a weekly review, with Claude doing the judgement work and Python doing the guarantees.

Owner & developer: **Wouter Altepost**

Version **v1** — released 14 September 2026

Document date: **17 September 2026**

Table of Contents

01	Executive Overview	3
02	System Architecture	5
03	Technology Stack	7
04	Data Model	9
05	Product Modules	11
06	AI Integration	16
07	Background Jobs	17
08	API Conventions & Auth	18
09	Design System & Branding	19
10	Deployment & Infrastructure	21
11	Engineering Process & Quality	22
12	Roadmap (v2 Backlog)	24
13	Appendix	25

This document was compiled from the Pozzy repository ([WouterAltepost/Pozzy](#)), its `docs/` folder, and the live Railway deployment, on 17 September 2026. It reflects the state of the v1 release plus the design pass that followed it.

Executive Overview

Pozzy is a single-user, private personal operating system: one web app that holds everything Wouter needs to run his week — calendar, mail triage, tasks, daily and weekly goals, habit tracking, hours per life area, study deadlines, quick capture, and a weekly review. Claude is used where judgement is needed — email priority, scheduling suggestions, a daily briefing, natural-language capture, weekly reflection — never for plain CRUD.

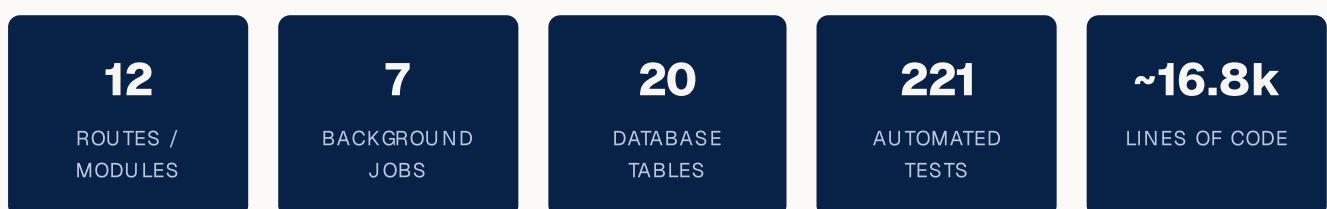
Purpose

Everything Wouter needs to run a week in one screen, with AI as an assistant that ranks and drafts but never invents. Free slots are computed deterministically; Claude only orders them. Daily “do’s” roll over automatically. Mail is triaged into one inbox but never written back to the server. Success is defined narrowly: Wouter can run a full day on Pozzy with no manual edits and knows within seconds what today needs.

Product Principles

- **The task is the interface.** Every screen answers “what needs my attention now” before anything else.
- **Honest state.** Synced, not synced, failed, empty and loading are all visible and specific — never hidden behind a spinner.
- **One vocabulary.** The same button, input, badge and card everywhere; brand lives in details, not decoration.
- **Data colour is data.** Area and mail-account colours come from the database and are never restyled; brand red/navy is reserved for identity.
- **Works without AI.** The design never depends on a Claude call or a sync to look complete — if Gmail is down, Tasks still work.

At a Glance



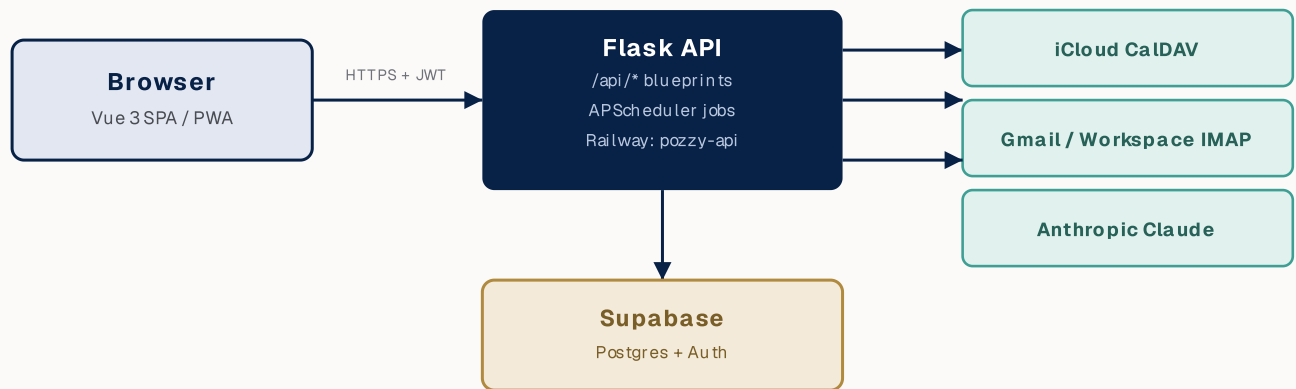
Life Areas

Every task, hour log, tracker and note is tagged to one of four life areas, each with its own colour stored in the database: **Study**, **Work** (alpaca AI, UDefine, and whatever else comes up), **Personal** (trading, investing, learning), and **Health**.

Single user by design. There is no `user_id` on any table in v1 — Pozzy is built for one account. The Supabase Auth user id is stored in Settings so row-level security can be added later if that ever changes.

System Architecture

Pozzy is a monorepo with two deployable services and one managed database. The Vue SPA never talks to Supabase directly for data — every read and write goes through the Flask API, which is the only thing holding business logic, secrets, and the three outbound integrations.



Two Railway services from one monorepo — api/ and web/

Repository Layout

```

pozzy/
  api/      Flask app, SQLAlchemy models, Alembic migrations, jobs, integrations
  web/      Vue 3 app (views, components, stores, api client)
  docs/     plan, ADR-style notes, prompt templates, design screens
  Branding/ logo and wordmark source files, embedded as-is
  CLAUDE.md conventions and hard rules for Claude Code sessions
  
```

Why SQLAlchemy over the Supabase client

All logic — rollover, scheduling, classification — lives in one place (the Flask API), and the frontend never needs the Supabase service key. Supabase is used purely as managed Postgres and Auth; nothing more. That keeps the door open to swapping it for plain Railway Postgres later without touching the frontend.

Why IMAP over the Gmail API

Google OAuth apps in “Testing” status issue refresh tokens that expire every 7 days, and `gmail.readonly` is a restricted scope that formally requires app verification for external production apps — a lot of process for a one-person tool. IMAP with a per-account app password (2-step verification required on each Google account) gives read access with no OAuth at all, works identically for the Workspace account, and

read-only is enforced simply by the client never issuing a write command. If richer features (labels, threads, replying) are wanted later, the Gmail API can be layered in for v2.

Technology Stack

Backend — `api/`

LAYER	CHOICE	NOTES
Language / runtime	Python 3.12	Pinned via <code>.python-version</code>
Framework	Flask ~3.1	App-factory pattern, one blueprint per module
ORM / migrations	SQLAlchemy ~2.0, Alembic ~1.16 (Flask-Migrate)	All schema changes go through a migration; never edit the DB by hand
DB driver	psycopg[binary] ~3.2	Against Supabase's Postgres transaction pooler
Auth verification	PyJWT[crypto] ~2.10	Verifies Supabase-issued ES256 JWTs via JWKS on every request
Scheduler	APScheduler ~3.11	In-process; 7 idempotent jobs, see Section 07
Calendar	caldav ~1.4, icalendar ~6.1, recurring-ical-events ~3.4	iCloud CalDAV client + recurrence expansion
Mail	imap-tools ~1.10	Read-only IMAP client, never issues STORE/COPY/MOVE/EXPUNGE
AI	anthropic ~0.84, pydantic ~2.9	One <code>claude_client.py</code> , JSON-schema-validated responses
Server / hosting	gunicorn ~23.0 on Railway	<code>--workers 1 --threads 4</code> (single worker, because the scheduler is in-process)
Testing	pytest ~8.4	221 tests across 5 stream-owned suites

Frontend — `web/`

LAYER	CHOICE	NOTES
Framework	Vue 3.5 (Composition API, <code><script setup></code>)	One route per module under <code>views/</code>
Build tool	Vite 7	<code>npm run build</code> only — no <code>npm ci</code> in the Railway build command
State	Pinia 3	One store per module

Routing	vue-router 4.5	Fixed route names referenced by widgets via <code>RouterLink</code>
PWA	vite-plugin-pwa 1.3	Network-first <code>/api</code> , cache-first assets, precached app shell
Icons	@phosphor-icons/vue 2.2	
Auth client	@supabase/supabase-js 2.49	Auth only — never used for data access
Styling	Plain CSS, design tokens in <code>style.css</code>	No component library; see Section 09
Typography	Geist (variable, weights 300–800)	Self-hosted <code>.woff2</code> , no runtime Google Fonts request

Infrastructure

PIECE	CHOICE
Hosting	Railway — two services (<code>api</code> , <code>web</code>) from one GitHub repo, root-directory routing
Database	Supabase Postgres, accessed only from Flask via SQLAlchemy
Auth	Supabase Auth — single account, email + password
Source control	GitHub — <code>WouterAltepost/Pozzy</code>
Built with	Claude Code, in parallel git-worktree sessions (Section 11)

Data Model

Postgres, via SQLAlchemy. Every table carries `id` (uuid), `created_at` and `updated_at`. Eight Alembic migrations take the schema from the M1 scaffold to the current 20-table shape, one migration per build stream plus a merge revision.

TABLE	PURPOSE
areas	The four life areas: name, colour, sort order
tasks	Title, area, tags, urgent/important flags, status, due date, estimate, scheduled window, linked calendar UID, source
daily_dos	The three (or more) do's for a date, done flag, rollover tracking
weekly_goals	Goals for a Monday-starting week, target/current value, done flag
trackers	Habit definitions — <code>daily_bool</code> , <code>weekly_count</code> , numeric, or duration type
tracker_entries	One value per tracker per date
hours_logs	Minutes logged per area/tag/task, optional note
calendar_accounts	iCloud CalDAV connection details, selected calendars
calendar_events	Local mirror of iCloud events, keyed on CalDAV UID + etag
mail_accounts	The four IMAP accounts, last-seen UID, colour, enabled flag
emails	Synced headers + snippet, Claude classification, manual overrides, handled flag
captures	Raw quick-capture text, parsed type/fields, processing status
notes	Markdown notes with area, tags, pinned flag
courses / deadlines / applications	Study courses, their deadlines (auto-linked to a task), internship application kanban
weekly_reviews	Sunday stats snapshot, Claude reflection, Wouter's notes, next-week focus
settings	Key/value store — working window, timezone, hour targets, AI kill switches, briefing time
ai_calls	Every Claude call: feature, model, tokens, cost estimate, ok/error
job_runs	Every background job execution: name, started, finished, ok, message

Dates. The API always returns ISO 8601 with timezone. Postgres stores UTC; the API layer converts using the timezone in Settings (`Europe/Amsterdam` , default).

Product Modules

Twelve guarded routes, each usable with zero AI and zero integrations — AI and syncs enrich a module, they are never required to open it.

1 Home

A ten-widget grid in a fixed order: three do's, today's events, top unhandled emails, tasks due or overdue, tracker quick-tick row, hours this week vs. target, weekly goals progress, upcoming study deadlines, the daily briefing (with an inline reply/reflection thread), and AI spend this month.

2 Agenda

Week and day views of iCloud events with scheduled tasks overlaid in a distinct colour. Create, edit and delete write back to iCloud via CalDAV. Recurring occurrences are read-only in v1 (editing one returns `409 recurring_not_editable`). Sync-now button and a visible last-synced timestamp.

3 Tasks

Due-date list with urgent/important chips computed from the Eisenhower matrix. A quick-add bar; “suggest a slot” computes free slots deterministically over the next 7 working days and asks Claude to rank the top 3 with a reason — accepting writes the event straight to iCloud.

4 Goals

Current week's goals with progress, plus today's and tomorrow's three do's. A 00:05 job rolls any undone do from yesterday onto today; a do rolled twice shows a warning badge. Claude can optionally suggest three do's from open urgent/important tasks and deadlines.

5 Mail

Unified inbox across four accounts, sorted by effective priority then date. Filters by account, category, area and needs-reply. Haiku classifies every new message into priority, category, area, needs-reply and a one-line summary; manual overrides never get clobbered by re-classification. Strictly read-only — “handled” is local state only.

6 Tracking

Habits grouped by area. Weekly Monday–Sunday grid for boolean and count trackers (e.g. daily creatine, 2–3 vitamins/week, weekly sauna); numeric and duration trackers get an 8-week line chart. Streaks and weekly completion percentage.

7 Hours

Minutes logged per area/tag, optionally linked to a task, plus a browser-side start/stop timer that creates a log on stop. Weekly table of totals per area against Settings targets.

8 Capture

One text box reachable from every page via the top bar. Submitting stores the raw text immediately; Claude then proposes what it is — task, event, goal, note or tracker entry — with parsed fields, and Wouter confirms or edits before anything is created. This is also the endpoint a future Telegram bot will hit.

9 Study

Courses with period and ECTS; deadlines auto-create a linked task that flips **urgent** on 3 days out. Internship application tracker as a kanban (found → applied → interview → offer/rejected) with next-step dates surfaced on Home.

10 Notes

Markdown notes with area and tags, pinned notes on top. Deliberately minimal — not a second brain.

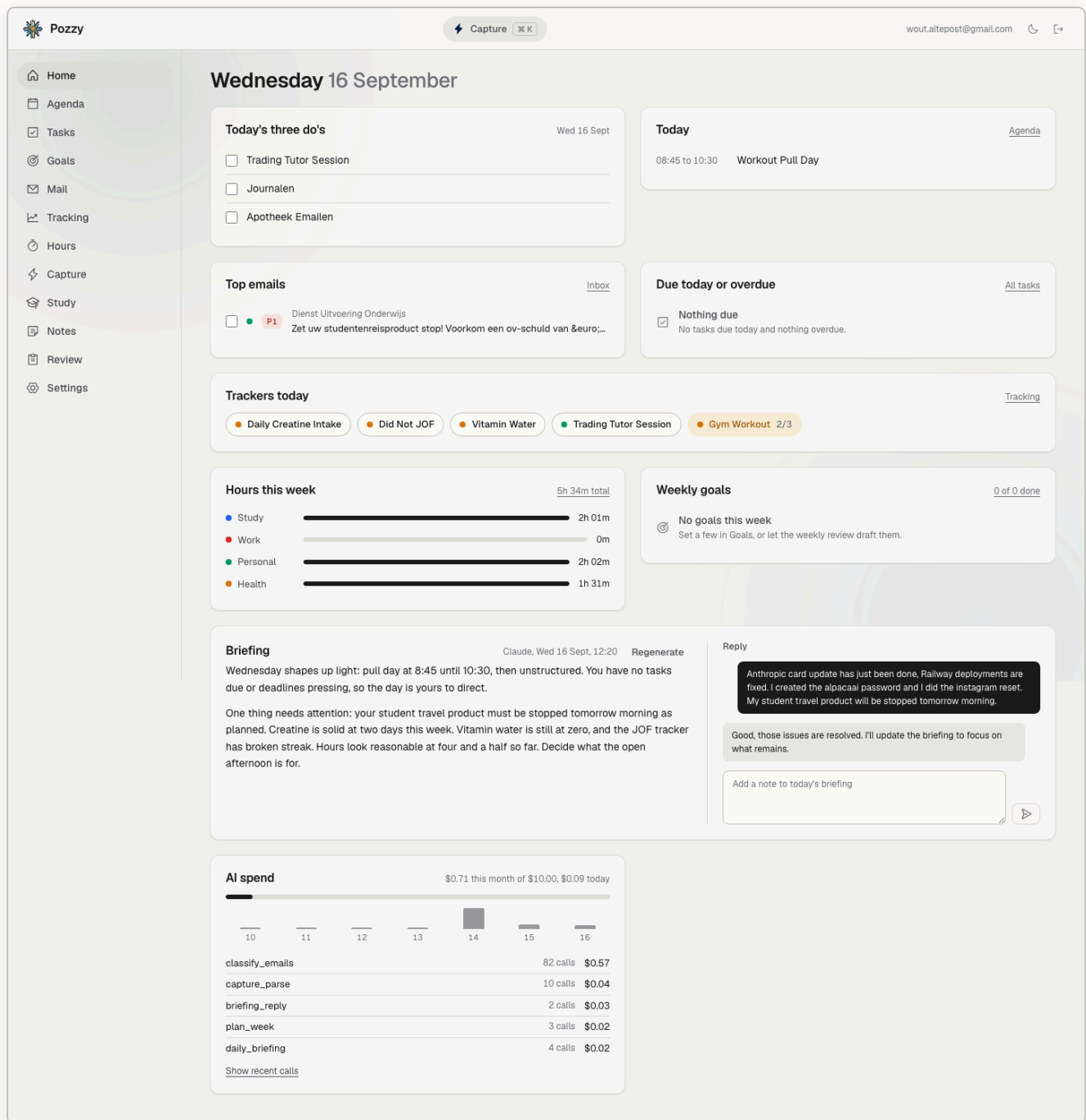
11 Weekly Review

A Sunday 18:00 job computes a stats snapshot (goals hit/missed, do completion rate, tracker completion, hours vs. target, tasks done/rolled, emails handled), Claude drafts a reflection and next week's focus, Wouter edits and finalises — which creates next week's goals.

12 Settings

Working window, timezone, hour targets per area, calendar and mail account management with connection tests, AI kill switches per feature, briefing time, and model selection. Also where job run history is visible when something fails.

Module Screenshots



Home — the ten-widget daily overview, light theme, desktop

Pozzy

Capture anything

wout.altepost@gmail.com

Home

Agenda

Tasks

Goals

Mail

Tracking

Hours

Capture

Study

Notes

Review

Settings

Agenda

Week of Mon 14 Sept

<

>

15/09/2026

Week

Day

New event

Synced Tue 15 Sept, 10:00

Sync now

	Mon 14	Tue 15	Wed 16	Thu 17	Fri 18	Sat 19	Sun 20
all day							
06:00							
07:00							
08:00							
09:00	08:45 Workout + Sauna	09:30 Sprint 1 NGS Projectb...	08:45 Workout Pull Day	09:30 Sprint 3 NGS Colleges...	08:45 Workout Push Day		
10:00							
11:00	11:00 Development van Pozzy						
12:00		12:30 Sprint 2 NGS Projectb...		12:30 Sprint 4 NGS MastercL...			
13:00							
14:00	14:00						
15:00	14:30 Boek aflezen		15:00 Kleren kopen			15:00 Sparta Weddie met C	15:30 FIT avv 5 - ASV Arsen...
16:00	15:45 Pozzy dev	16:45 Zu aan I&T zitten					
17:00							
18:00							
19:00							
20:00		20:00 Training		20:00 Bios			
21:00							
22:00	21:30					21:00 Baar BDay Rijnbar	

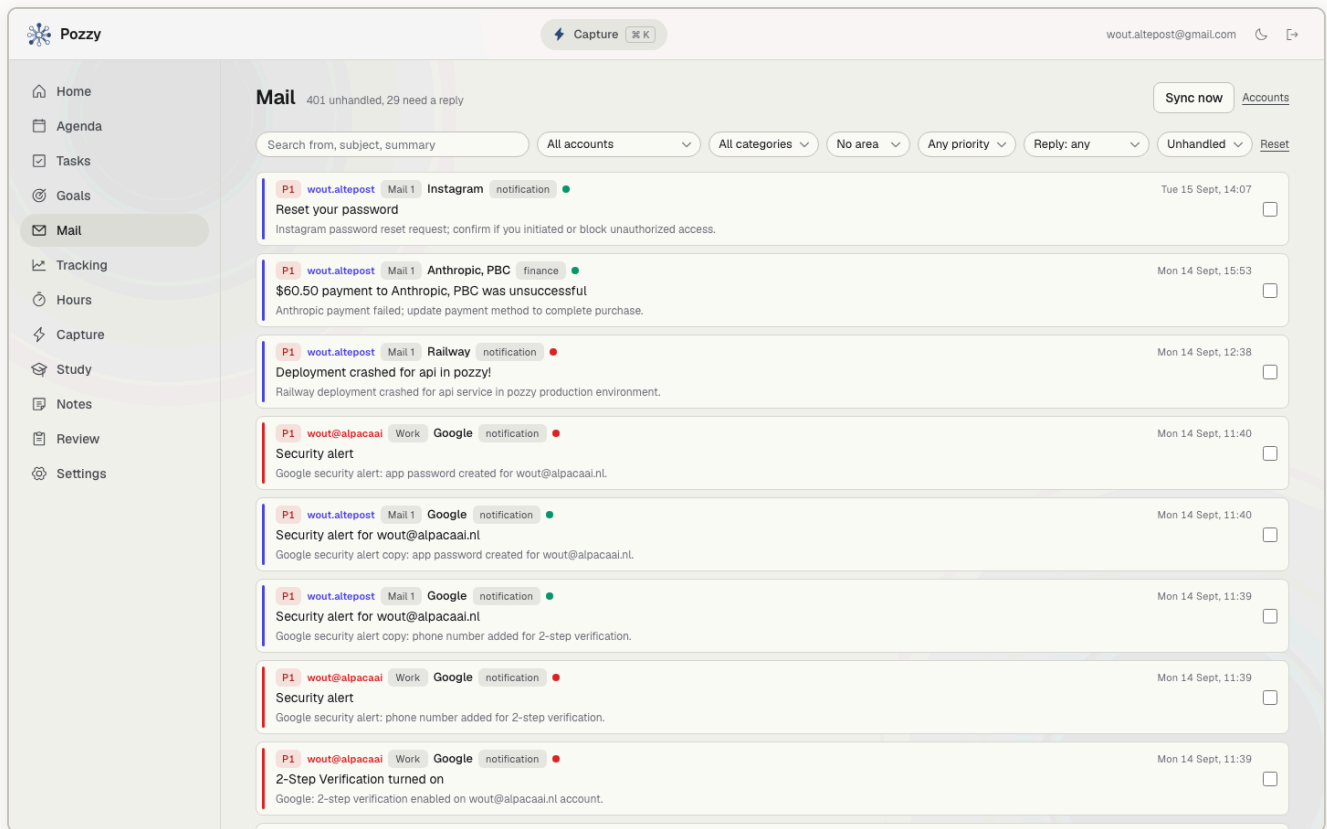
event

recurring

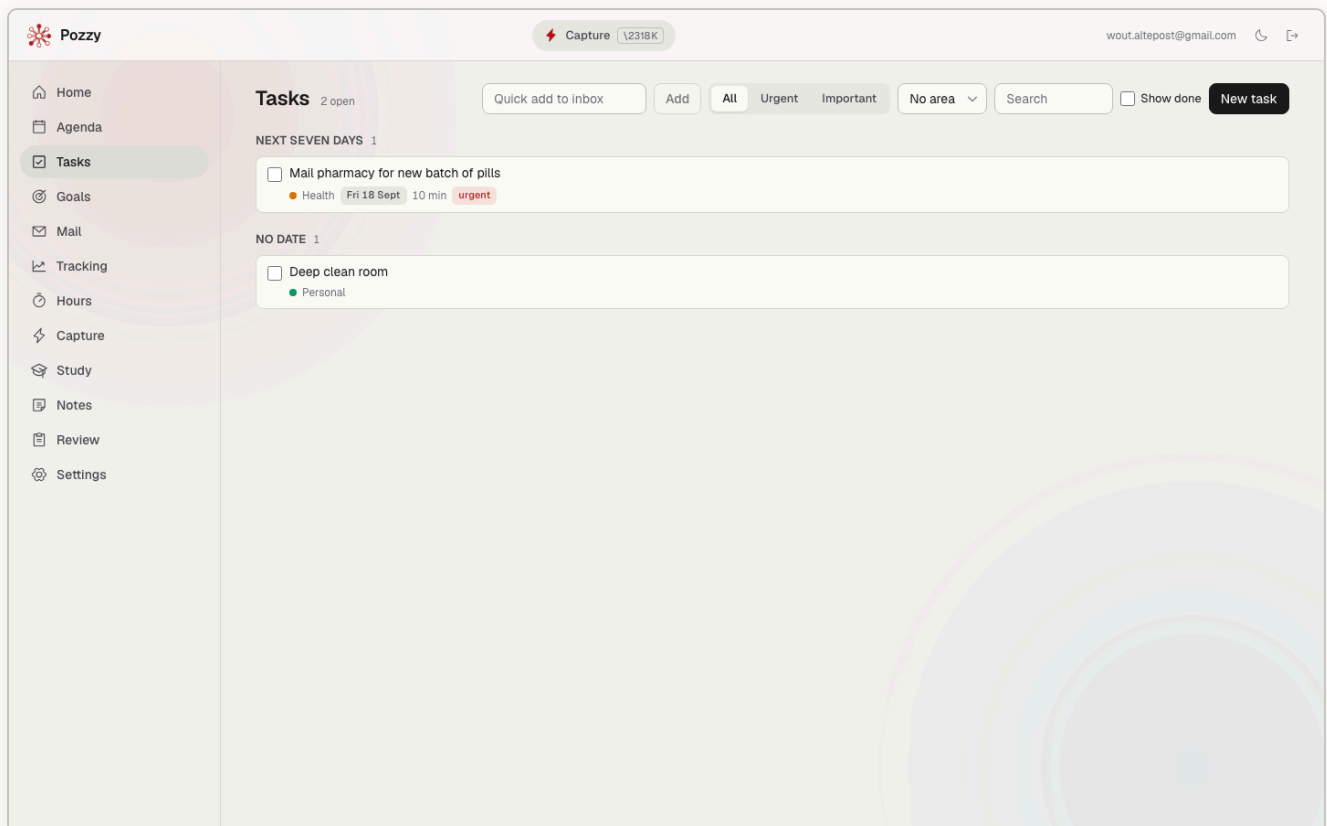
Pozzy task event

scheduled task (not yet on iCloud)

Agenda — week view over the synced iCloud mirror



Mail — unified inbox across four accounts, Claude-classified priority



Tasks — due-date list with Eisenhower chips and the slot-suggestion panel

AI Integration

Every Anthropic call goes through one function, `api/app/integrations/claude_client.py`, uses a named prompt template from `docs/prompts/`, and writes a row to `ai_calls` with model, token counts and an estimated cost — so spending is always visible on the homepage. Each feature has its own kill switch in Settings, checked before the client is ever built.

FEATURE	MODEL	TRIGGER
Email classification	Haiku	Every 15 minutes, new mail only, batched 10–20 per call
Daily briefing	Sonnet	07:00 job, or manual regenerate from the widget
Scheduling suggestion	Sonnet	On demand, per task (“suggest a slot”)
Natural-language capture	Sonnet	On submit from the Capture bar
Weekly reflection	Sonnet	Sunday 18:00 job

The scheduling guarantee

Free-slot computation is deterministic Python: the working window, minus calendar events, minus already-scheduled tasks, honouring an estimated duration — producing a shortlist of about 12 candidates. Claude is only ever given that shortlist to rank and explain. A bad model answer can shuffle the ranking; it can never invent a slot that doesn’t exist.

Graceful degradation

Every module keeps working with AI switched off. Mail falls back to rule-based classification (notifications and newsletters get low priority; finance/school/client/personal by sender and subject heuristics). Capture falls back to a deterministic parser recognising prefixes (`note:`, `goal:`, `event:`), tracker names, relative dates and hashtags. The briefing and weekly review fall back to deterministic text built straight from the data.

Cost discipline. A test run of all prompts against the live models cost about five US cents for eleven calls; day-to-day email classification runs to cents per day. `ai_cost_rollup` stores a 90-day daily spend history in Settings, and Settings carries a monthly budget the homepage widget tracks against.

Background Jobs

Registered in `api/app/scheduler.py`, running in-process under APScheduler on the API service (in the app timezone, Europe/Amsterdam). Every job wraps its own session, catches every exception, and writes a `job_runs` row via a shared `run_job` helper — a job failure is visible in Settings and never crashes the web process.

JOB	SCHEDULE	WHAT IT DOES
<code>three_do_rollover</code>	00:05 daily	Copies yesterday's undone do's to today, idempotent per day
<code>deadline_urgency</code>	00:10 daily	Flips <code>urgent</code> on tasks due within the configured window (default 3 days)
<code>calendar_sync</code>	every 10 min	Mirrors selected iCloud calendars into <code>calendar_events</code> ; one failing calendar doesn't block the rest
<code>mail_sync_and_classify</code>	every 15 min	Incremental IMAP sync (7-day backfill on first run) then Haiku classification; one failing account doesn't stop the others
<code>daily_briefing</code>	07:00 (configurable)	Writes the day's briefing via Claude or deterministic text; idempotent, Regenerate forces a rewrite
<code>weekly_review_draft</code>	Sunday 18:00	Computes the week's stats snapshot and drafts the reflection and next week's focus
<code>ai_cost_rollup</code>	23:55 daily	Rolls today's AI spend per feature into a 90-day Settings history

Why one gunicorn worker. APScheduler runs in-process, so the API service is pinned to `--workers 1 --threads 4` — more workers would run every job multiple times in parallel. Moving the scheduler to its own Railway service is the documented path to unlocking more workers (v2 backlog).

API Conventions & Auth

Structure

REST, JSON, everything under `/api`. One blueprint per module, each with `models.py`, `schemas.py`, `routes.py` and `service.py`. Standard CRUD per resource plus explicit action endpoints where a request does more than persist a record, e.g. `POST /tasks/{id}/suggest-slot`, `POST /calendar/sync`, `POST /reviews/{week}/finalize`.

Response envelope

```
Success — { "data": ..., "error": null }
Failure — { "data": null, "error": { "code", "message" } }
```

Authentication

Supabase Auth issues an ES256-signed JWT to the Vue SPA on login. Every `/api/*` route validates that token via `Authorization: Bearer`, verified against Supabase's JWKS — with one deliberate exception, `/api/health`. The Vue app never sees the Supabase service key or the raw `DATABASE_URL`.

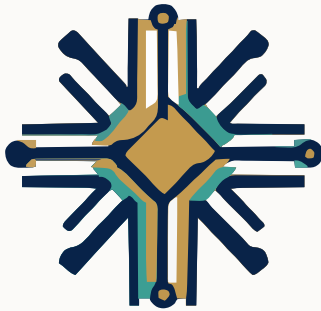
Engineering hard rules

Ten rules in `CLAUDE.md` bound every change to the API, enforced by convention rather than tooling:

1. Every module works with AI and integrations switched off.
2. All business logic lives in Flask; Vue only calls `/api/*`.
3. Every route validates the Supabase JWT except `/api/health`.
4. Secrets only via environment variables, never committed.
5. Background jobs are idempotent and log to `job_runs`; a failure never crashes the web process.
6. Every Anthropic call goes through `claude_client.py`, uses a named prompt, logs to `ai_calls`, and has a kill switch.
7. Free slots are computed deterministically; Claude only ranks candidates it is given.
8. Mail is read-only — the IMAP client never issues a write command.
9. Schema changes go through Alembic; never edit the database by hand.
10. Finish and deploy a milestone before starting the next one.

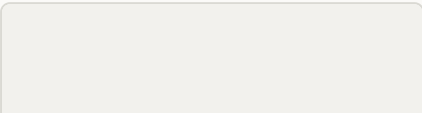
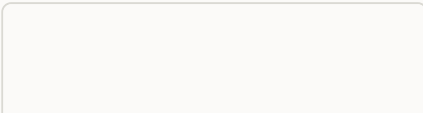
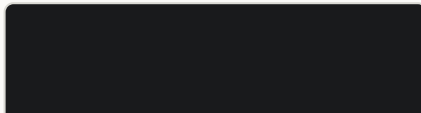
Design System & Branding

Name: **Pozzy**. Mark: a hand-inked navy hub with eight spokes ending in rings, rendered over a gold diamond core with teal accents. The mark shipped in three versions before landing on the current one, and every asset in `Branding/` is treated as final — embedded exactly as delivered, never redrawn, restyled or recoloured.



Mark history: the original export was solid red (`#AF1616` / `#A81010`), briefly recoloured to “galaxy blue” (`#2A4B7C`) on 15 September, then replaced entirely on 16 September with the current navy / gold / teal / white mark shown here. The brand colour token follows the current mark’s navy, `#082247`.

Colour Tokens (light theme)

		
Brand <code>#082247</code>	Gold accent <code>~#B08A3E</code>	Teal accent <code>~#3E9F94</code>
		
Background <code>#F2F1ED</code>	Surface <code>#FBFAF8</code>	Ink <code>#191A1C</code>
		
Success <code>#1F7A4D</code>	Warning <code>#A35E08</code>	Danger <code>#B3261E</code>

A full dark-theme token set mirrors every value above (e.g. brand becomes `#8AA6D9` on a `#141518` background) and applies automatically via `prefers-color-scheme`, overridable per-user with a top-bar toggle.

Typography & Layout

Body and UI text is set in **Geist** (variable weight 300–800), self-hosted as a single `.woff2` — there is no runtime request to Google Fonts. Type scale runs 11px to 28px (`--fs-xs` to `--fs-3xl`); spacing is a 4px-based scale from 4px to 40px; corner radii run 6px to a full pill at 999px. The top bar is a fixed 56px, the desktop nav rail is 232px, and content tops out at 1180px wide.

Data colour vs. brand colour

The four area colours and each mail account's colour are stored in the database and rendered as-is throughout the app — they are data, not decoration, and are never restyled to fit the palette. Brand navy and gold are reserved for identity and the “now” marker on the agenda.

Accessibility

WCAG AA contrast in both light and dark, a visible keyboard focus ring on every control, and reduced-motion behaviour for every transition. Every view is usable at 390px width with no horizontal scroll — the primary phone target is an installed PWA on iPhone.

Deployment & Infrastructure

Live environment

SERVICE	URL
Web (Vue SPA)	web-production-e418d.up.railway.app
API (Flask)	api-production-c9d96.up.railway.app

Topology

Two Railway services from one GitHub repository (`WouterAltepost/Pozzy`), routed by root directory (`api`, `web`). The API builds via its `Procfile` (`gunicorn "app:create_app()" --bind 0.0.0.0:$PORT --workers 1 --threads 4`) with a `/api/health` healthcheck; the web service runs `npm run build` then serves the static bundle with `serve -s dist`.

Database connection

Alembic migrations and application traffic both go through Supabase's **transaction pooler** (`aws-1-eu-west-1.pooler.supabase.com:6543`) rather than the direct host, which is IPv6-only and unreliable from Railway. Scheduled jobs needed prepared statements disabled to work through that pooler without psycopg errors.

Hard-won deployment notes

- Railway's web build collided `npm ci` with Railpack's `node_modules` cache mount; the build command is `npm run build` only.
- `MAIL_ACCOUNTS_JSON` must be set from the `.env` file directly, never through a shell — quoting mangles the JSON otherwise. Verified by hash after setting.
- `WEB_ORIGIN` must match the browser origin exactly (scheme + host, no path, no trailing slash) or requests fail CORS in the browser while still succeeding from curl.
- `VITE_*` values are baked in at build time; changing one needs a redeploy of `web`.
- Commits touching only `docs/` don't trigger a deploy, because of the root-directory routing.

Required environment & secrets

Supabase project (URL, anon key, JWT secret, pooled connection string) · Anthropic API key · Apple ID + app-specific password (iCloud CalDAV) · four Google app passwords (three Gmail, one Workspace, each requiring 2-step verification) · a generated Flask secret key · `TZ=Europe/Amsterdam`.

Engineering Process & Quality

How v1 was built

After a scaffold milestone (M1) deployed end-to-end, the rest of v1 was built as **five parallel Claude Code sessions** running in separate git worktrees, each owning a stream with its own file set, its own Alembic migration, and its own docs registry entry in `docs/PROGRESS.md`:

STREAM	SCOPE
A — Local modules	Areas, Settings, Tasks, Goals, Trackers, Hours, Notes, Study, Home v1
B — Calendar	CalDAV read sync, Agenda views, then write
C — Mail	IMAP sync, Haiku classification, unified inbox
D — AI	Claude client, Capture, Briefing, Weekly review, Spend tracking
E — Integration	Routes, widgets, scheduler registration, merging all four streams

Streams A and B ran first, then C and D, with E merging last — each stream documented its contract in `AI_CONTRACTS.md`, `ROUTES.md`, `WIDGETS.md` and `JOBS.md` so the integration stream could wire everything together with only routine rebase conflicts.

v1 shipped 14 September 2026, evening. All four streams merged, the full test suite green, the PWA served from Railway, the scheduler running in production, and an iPhone round trip of an Agenda-created event confirmed by Wouter.

The design pass

On 15–16 September, a single Claude Code session applied a presentation-only design brief (`docs/briefs/DESIGN.md`) against the working app — no API, store, router, or data-model changes allowed, shared components and tokens only, never per-view one-off styling. It produced the full light/dark, desktop/mobile screenshot set in `docs/design-screens/` used throughout this document, and replaced the mark three times before landing on the current navy/gold/teal version (Section 09).

Testing

221 automated tests run under pytest, organised by the stream that owns the code: free-slot engine and three-do rollover (A), CalDAV sync and recurrence parsing (B), IMAP parsing and mail classification (C), the Claude client contract and every AI feature (D), and cross-stream integration — the scheduler, mail-account env parsing, briefing notes (E). Smoke scripts (`caldav_smoke.py`, `imap_smoke.py`, `claude_smoke.py`) were run against the real accounts and the real Anthropic API before merge, not just mocked in CI.

Repository size

~8.7k

LINES — API/APP

~8.2k

LINES — WEB/SRC

8

ALEMBIC MIGRATIONS

70

COMMITTS SINCE 14
SEP

Roadmap — v2 Backlog

Nothing below blocks daily use of v1; all of it was deliberately deferred, either by the original plan or by a build stream running out of scope for the milestone.

Deferred by the original plan

- Sending or replying to email; Gmail API and labels beyond what `X-GM-LABELS` already exposes.
- A Telegram bot for capture and notifications — `POST /api/captures` is already the endpoint it will hit.
- Multi-user or shared use; row-level security is not enabled.
- Trading 212 or finance widgets, a native iOS app, offline-first PWA sync, recurring tasks (trackers cover recurring habits today).

Found while building

- A server-side hours timer — v1 keeps it in browser `localStorage`, which a Telegram-driven start/stop can't reach.
- Editing a single occurrence of a recurring iCloud event (currently returns `409`).
- Moving APScheduler to its own Railway service to unlock more than one gunicorn worker.
- An end-to-end browser test suite — v1 was verified manually and via targeted smoke scripts, no Playwright suite exists yet.
- Push notifications for the briefing and deadline alerts.

Deferred by the design pass

- Replacing eight remaining `window.prompt` / `window.confirm` call sites with inline UI — a control-flow change, out of scope for a presentation-only pass.
- Reviewing the seeded Work area colour, which currently sits close to the old brand red.

Appendix

Route Map

#	PATH	NAME	NAV LABEL
1	/	home	Home
2	/agenda	agenda	Agenda
3	/tasks	tasks	Tasks
4	/goals	goals	Goals
5	/mail	mail	Mail
6	/trackers	trackers	Tracking
7	/hours	hours	Hours
8	/capture	capture	Capture
9	/study	study	Study
10	/notes	notes	Notes
11	/review	review	Review
99	/settings	settings	Settings

Key environment variables

VARIABLE	USED BY
DATABASE_URL	Flask — Supabase Postgres, transaction pooler
SUPABASE_URL / SUPABASE_JWT_SECRET	Flask — JWT verification; Vue — Auth client
ANTHROPIC_API_KEY, CLAUDE_MODEL_FAST/SMART	Flask — <code>claude_client.py</code>
ICLOUD_USERNAME / ICLOUD_APP_PASSWORD	Flask — CalDAV client
MAIL_ACCOUNTS_JSON	Flask — four IMAP accounts
SECRET_KEY, TZ, WEB_ORIGIN	Flask — app config, CORS
VITE_SUPABASE_URL / VITE_SUPABASE_ANON_KEY / VITE_API_BASE	Vue — baked in at build time

Source documents

Compiled from `docs/POZZY_PLAN.md`, `PRODUCT.md`, `CLAUDE.md`, `docs/ROUTES.md`, `docs/WIDGETS.md`, `docs/JOBBS.md`, `docs/V2.md`, `docs/DEPLOY.md`, `docs/PROGRESS.md`, `docs/AI_CONTRACTS.md`, the `api/` and `web/` source trees, and the live Railway deployment.

Pozzy · Personal Operating System · built by Wouter Altepost with Claude Code · this document generated 17 September 2026.